

Contents

08 — Plano de Ação: Universalidade Total entre Harnesses (REGRA 10)	1
1. Contexto e objetivo	1
2. Arquitetura de 3 camadas (aplicada)	2
3. Decisões de design	2
4. Evidência (guarda verde)	3
5. Manutenção futura	3

08 — Plano de Ação: Universalidade Total entre Harnesses (REGRA 10)

Status: concluído (2026-08-05 — guardas determinísticos verdes: verificar-universalidade.py e verificar-consistencia-pipeline.py ambos --estrito) **Data:** 2026-08-05 **Escopo:** arquitetura de 3 camadas (canônico único + adaptadores finos por harness + rules finas por harness), cobrindo comando, skill, MCP, hook, SPEC, config e regra **Impacto em scripts Python:** adicionado scripts/verificar-universalidade.py (novo árbitro determinístico — nenhum script existente foi alterado)

1. Contexto e objetivo

O operador demandou uma regra canônica: **qualquer skill, MCP, hook, SPEC, config ou qualquer personalização do repositório deve funcionar em qualquer harness** (Claude Code, opencode, Gemini CLI, CodeBuddy, Qoder, etc.).

Motivação prática: `/.claude/commands/*.md` é mecanismo de descoberta exclusivo do Claude Code — o opencode **não o lê** (usa `.opencode/commands/*.md` ou a chave `command` do `opencode.jsonc`). Sem a camada de adaptadores, comandos universais por conteúdo (`SPEC_COMANDOS.md`) permaneciam invisíveis em outros harnesses, e nada impedia que uma nova personalização nascesse “só para Claude Code”.

2. Arquitetura de 3 camadas (aplicada)

Camada	O que é	Onde vive
1. Canônico único	Procedimentos, regras, design, skills — fonte de verdade, sem lógica duplicada em arquivo de harness	SPEC_COMANDOS.md, AGENTS.md, SPEC.md, brand/*, .claude/skills/*/SKILL.md (padrão aberto Agent Skills)
2. Adaptadores finos de descoberta	Mecanismos nativos de cada harness que apenas apontam para o canônico	.claude/commands/*.md (Claude Code), .opencode/commands/*.md (opencode), .mcp.json e opencode.jsonc (MCP), .claude/settings.json (hooks proprietários)
3. Rules finas por harness	Arquivo de instruções do harness referencia AGENTS.md + SPEC_COMANDOS.md sem duplicar listas/regras	CLAUDE.md, GEMINI.md, CODEBUDDY.md, QODER.md

Hooks e automações nativas são **conveniência, nunca requisito**: o gate de qualidade é sempre o script determinístico, listado como **passo manual obrigatório** no SPEC_COMANDOS.md (novo bloco “Guardas determinísticos”).

Fallback universal para qualquer harness sem adaptador nativo: mapeamento por **linguagem natural** (pedido equivalente descrito no SPEC_COMANDOS.md).

3. Decisões de design

1. **Novo árbitro scripts/verificar-universalidade.py** com --estrito (exit 1 se lacuna). Checa, deterministicamente:

- (a) **Comandos**: cada seção ## /comando do SPEC_COMANDOS.md tem adaptador em .claude/commands/ e em .opencode/commands/; cada adaptador referencia SPEC_COMANDOS.md, tem frontmatter --- com description, e cabe no teto de 3000 bytes (ponteiro fino — flag de “cópia do procedimento”, bug histórico de TIPOS_VALIDOS duplicado);
- (b) **Adaptadores órfãos**: todo .md em commands/ de qualquer harness corresponde a uma seção canônica (nada de comando esquecido);
- (c) **Rules**: CLAUDE.md/GEMINI.md/CODEBUDDY.md/QODER.md existem, referenciam AGENTS.md e SPEC_COMANDOS.md, e respeitam teto de 6000 bytes;
- (d) **Skills**: todo .claude/skills/*/SKILL.md tem frontmatter name+ description (padrão aberto Agent Skills — exigido por Claude Code, opencode e Gemini CLI);
- (e) **MCPs**: todo servidor declarado em .mcp.json existe em opencode.jsonc (hoje: code-review-graph em ambos);

- (f) **Hooks:** os dois guardas são citados no SPEC_COMANDOS.md como passo manual obrigatório (hooks proprietários nunca podem ser o único gatilho).
2. **.opencode/commands/ criado com 12 ponteiros finos** espelhando o .claude/commands/ — mesma disciplina: frontmatter description + corpo que manda ler a seção correspondente de SPEC_COMANDOS.md (\$ARGUMENTS = <slug>).
 3. **Rules adelgadas:** a lista de 12 comandos (e seus parâmetros) foi removida de CLAUDE.md, GEMINI.md, CODEBUDDY.md e QODER.md — agora referenciam AGENTS.md (arquitetura/regras) + SPEC_COMANDOS.md (procedimentos) e só descrevem o comportamento esperado. O bloco MCP de GEMINI.md/CODEBUDDY.md/ QODER.md foi preservado: é o mecanismo nativo desses harnesses para habilitar o MCP (camada 2, não regra de negócio).
 4. **REGRA 10 adicionada ao AGENTS.md** (“Universalidade Total — canônica, inegociável”) + bloco “Guardas determinísticos” no SPEC_COMANDOS.md.
 5. **Hook estendido:** o PostToolUse de .claude/settings.json agora também roda verificar-universalidade.py --estrito ao editar qualquer camada universal (AGENTS.md, harness rules, SPECs, .mcp.json, opencode.jsonc, settings.json, qualquer commands/*.md, qualquer skills/*/SKILL.md, o próprio guarda).

4. Evidência (guarda verde)

```
$ python scripts/verificar-universalidade.py --estrito
VERIFICACAO DE UNIVERSALIDADE ENTRE HARNESSES
[OK] Comandos, rules e skills universais: canonicos unicos + adaptadores finos
    em todos os harnesses suportados.    (exit 0)
```

```
$ python scripts/verificar-consistencia-pipeline.py --estrito
VERIFICACAO DE CONSISTENCIA DO PIPELINE - 9 tipo(s) de material
[OK] Todos os tipos de material estao presentes em todas as camadas.    (exit 0)
```

Durante o desenvolvimento do guarda, ele pegou 2 bugs do próprio guarda (comparação caminho.name vs p.stem; órfãos sem remoção de extensão) e 2 lacunas reais (guardas não citados no SPEC_COMANDOS.md) — todas corrigidas; o guarda se autoverificou até ficar verde.

5. Manutenção futura

- **Novo comando:** adicionar seção ## /nome no SPEC_COMANDOS.md + 2 ponteiros finos (.claude/commands/nome.md e .opencode/commands/nome.md) → rodar os 2 guardas.
- **Novo harness rules:** criar NOVOHARNESS.md referenciando AGENTS.md + SPEC_COMANDOS.md e adicioná-lo a HARNESS_RULES do guarda.
- **Novo MCP:** declarar em .mcp.json e opencode.jsonc — o guarda falha se só um dos dois tiver.
- **Procedimento novo que só existiria como hook:** escrever o passo manual no canônico; o hook é opcional.